

Le flag `--dangerously-skip-permissions`

Ce flag désactive les confirmations interactives de Claude Code, ce qui est indispensable pour les pipelines CI/CD non interactifs. Mais il supprime aussi la dernière ligne de défense contre les actions destructives. Son utilisation n'est acceptable que dans un container éphémère isolé, jamais sur une machine partagée persistante.

```
# Acceptable en CI (container éphémère)
--dangerously-skip-permissions \
"Run tests and fix failing ones"

# Inacceptable (machine partagée, accès réseau libre)
claude --dangerously-skip-permissions # ❌
```

La règle : le flag élargit la confiance accordée à Claude, donc le périmètre du container doit compenser en réduisant ce que Claude peut atteindre.

Environnement recommandé

Un container Docker jetable créé au début du job et détruit à la fin. Le dépôt est monté en volume, les dépendances installées à l'intérieur, et aucun fichier sensible de la machine hôte n'est accessible.

```
jobs :
  agent-task :
    runs-on : ubuntu-latest
    container :
      image : node:22-alpine
    steps :
      - uses : actions/checkout@v4
      - name : Run Claude
        env :
          ANTHROPIC_API_KEY : ${{ secrets.ANTHROPIC_API_KEY }}
        run : |
          claude --dangerously-skip-permissions \
            -p "Run tests, fix failures, open PR"
```

Pour une isolation maximale, les solutions cloud (E2B, Vercel Sandboxes) offrent des microVMs avec un kernel séparé, ce qui élimine le risque d'échappement de container.

Whitelist d'outils stricte

Même en mode bypass, il est possible de limiter les outils disponibles. Un agent de review n'a besoin que de `Read`, `Glob`, `Grep`. Un agent d'implémentation peut avoir `Edit` sur des chemins ciblés, mais rarement `Bash` sans contrainte.

Rôle de l'agent	Outils autorisés
Review de code	Read, Glob, Grep
Correction de tests	Read, Edit, Bash(test runner)
Analyse de sécurité	Read, Glob, Grep
Implémentation	Read, Edit, Write, Bash

La whitelist se configure via `--allowedTools` en CLI ou `allowed_tools` dans l'action GitHub.

Secrets en CI

Les clés API et tokens ne doivent jamais apparaître dans le fichier YAML du workflow. Utiliser les secrets GitHub (Settings → Secrets and variables → Actions) et les injecter en variables d'environnement.

```
env :
  ANTHROPIC_API_KEY : ${{ secrets.ANTHROPIC_API_KEY }}
  # Jamais :
  # ANTHROPIC_API_KEY: sk-ant-xxxxx ❌
```

Rotation recommandée : changer les clés tous les 90 jours et auditer les accès via les logs GitHub.

Réseau : accès restreint

Ne jamais exécuter Claude en mode bypass avec un accès réseau non restreint. Configurer une allowlist réseau explicite pour les domaines légitimes :

```
api.anthropic.com # Modèle
*.npmjs.org # Packages
github.com # Git operations
```

Tout le reste en deny par défaut. Cela bloque l'exfiltration de données vers des endpoints externes si l'agent est compromis ou mal configuré.

Pattern safe autonomy

Le pattern le plus sûr pour la production : Claude propose, un humain approuve. L'agent ouvre une PR avec ses changements, la pipeline de tests valide, et un reviewer humain approuve avant le merge.

```
Agent → branche → PR → CI tests → human review → merge
```

Ne pas automatiser le merge final, même si tous les tests passent. Garder un humain dans la boucle pour les changements vers main.

Anti-patterns à éviter

Anti-pattern	Risque
Skip permissions sur machine hôte	Accès filesystem/réseau non borné
Monter <code>/home</code> dans le container	Exposition des clés SSH, credentials
Allowlist réseau <code>*</code>	Exfiltration possible
Merge automatique sans review	Régression non détectée